

WEEK 2 MODULE

1. Linear Search

Definition:- Linear Search is a Searching technique in which each element is checked one by one until the target element is found.

Algorithm: (Step by Step)

- (i) Start from the first element
- (ii) Compare each element with the target
- (iii) if found, stop Searching
- (iv) if the end is reached, the element is not present.

Time Complexity :- $O(n)$

Advantages:-

- (i) Easy to implement
- (ii) work on unsorted data

Code of linear SearchProblem 1: Fraud Transaction Detection

#include <iostream>

using namespace std;

int main() {

int n, target;

cin >> n;

int arr[n];

for (int i = 0; i < n; i++) {

cin >> arr[i];

}

cin >> target;

bool found = false;

for (int i = 0; i < n; i++) {

if (arr[i] == target) {

found = true;

break;

}

}

if (found) {

cout << "Fraud Transaction found";

} else {

cout << "Transaction Not found";

}

return 0;

}

G > Users > gungu > linear_search.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n, target;
6      cin >> n;
7
8      int arr[n];
9
10     for(int i = 0; i < n; i++)
11         cin >> arr[i];
12
13     cin >> target;
14
15     bool found = false;
16
17     for(int i = 0; i < n; i++) {
18         if(arr[i] == target) {
19             found = true;
20             break;
21         }
22     }
23
24     if(found)
25         cout << "Fraud Transaction Found";
26     else
27         cout << "Transaction Not Found";
28
29     return 0;
30 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\gungu> g++ linear_search.cpp -o linear_search

PS C:\Users\gungu> .\linear_search

5

1201 1305 1450 1408 1502

1450

Fraud Transaction Found

PS C:\Users\gungu> |

2. Binary Search

Definition:- Binary Search is a Searching technique used on sorted data. it repeatedly divides the search space into two half.

Algorithm: (Step by Step)

- (i) Find the middle element
- (ii) if target equal to middle element, return found
- (iii) if target is smaller, search in left half
- (iv) if target is greater, search in right half
- (v) Repeat until found or search space become empty.

Time Complexity : $O(\log n)$

Advantages:

- (i) faster than linear search
- (ii) Easy to search in larger data also

Code of Binary Search

Problem 2: Server log Search

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
```

```
    cin >> n;
```

```
    int arr[n];
```

```
    for(int i=0; i<n; i++){
```

```
        cin >> arr[i]
```

```
    }
```

```
    int target;
```

```
    cin >> target;
```

```
    int low = 0, high = n-1;
```

```
    bool found = false;
```

```
    while (low <= high) {
```

```
        int mid = (low + high) / 2;
```

```
    }
```

```
    if (arr[mid] == target) {
```

```
        found = true;
```

```
        break;
```

```
    }
```

```
else if (arr[mid] < target) {  
    low = mid + 1;  
} else {  
    high = mid - 1;  
}
```

```
if (found) {  
    cout << "log found";  
} else {  
    cout << "log Not found";  
}
```

```
return 0;  
}
```

C: > Users > gungu > binary_search.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      int arr[n];
9
10     for(int i = 0; i < n; i++)
11         cin >> arr[i];
12
13     int target;
14     cin >> target;
15
16     int low = 0, high = n - 1;
17     bool found = false;
18
19     while(low <= high) {
20         int mid = (low + high) / 2;
21
22         if(arr[mid] == target) {
23             found = true;
24             break;
25         }
26         else if(arr[mid] < target)
27             low = mid + 1;
28         else
29             high = mid - 1;
30     }
31
32     if(found)
33         cout << "Log Found";
34     else
35         cout << "Log Not Found";
36
37     return 0;
38 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\gungu> g++ binary_search.cpp -o binary_search

PS C:\Users\gungu> .\binary_search.exe

7

1001 1005 1010 1015 1020 1030 1040

1015

Log Found

PS C:\Users\gungu> |

3. Bubble Sort :-

Definition:- Bubble Sort repeatedly compare adjacent element and swaps them if they are in the wrong order.

Algorithm: (Step by Step)

- (i) Compare adjacent element
- (ii) Swap if needed.
- (iii) Repeat until array become sorted.

Time complexity : $O(n^2)$

Code of Bubble Sort

Problem: 3: Bubble Sort.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cin >> n;
```

```
    int arr[n];
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    }
```

```
    for(int i=0; i<n-1; i++) {
        for (j=0; j<n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
```

```
    for (int i=0; i<n; i++) {
        cout << arr[i] << " ";
    }
```

```
    return 0;
```

```
}
```

Welcome

linear_search.cpp

binary_search.cpp

bubble_sort.cpp X

C: > Users > gungu > bubble_sort.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      int arr[n];
9
10     for(int i = 0; i < n; i++)
11         cin >> arr[i];
12
13     for(int i = 0; i < n - 1; i++) {
14         for(int j = 0; j < n - i - 1; j++) {
15             if(arr[j] > arr[j + 1]) {
16                 swap(arr[j], arr[j + 1]);
17             }
18         }
19     }
20
21     for(int i = 0; i < n; i++)
22         cout << arr[i] << " ";
23
24     return 0;
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\gungu> g++ bubble_sort.cpp -o bubble_sort

PS C:\Users\gungu> .\bubble_sort.exe

5

45000 32000 55000 28000 60000

28000 32000 45000 55000 60000

PS C:\Users\gungu> |

4. Insertion Sort.

Definition :- Insertion Sort build the ~~the~~ sorted array one element at a time by inserting element into their correct position.

Algorithm: (Step by Step)

- (i) Assume first element is sorted
- (ii) Pick next element
- (iii) Insert it into correct position
- (iv) Repeat for all element

Time Complexity : $O(n^2)$

Code of Insertion Sort

Problem 4: Sort website Response time

```
#include <iostream>
using namespace std;
int main () {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```


Welcome linear_search.cpp binary_search.cpp bubble_sort.cpp insertion_sort.cpp X

C:\Users\gungu> g++ insertion_sort.cpp -o insertion_sort

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      int arr[n];
9
10     for(int i = 0; i < n; i++)
11         cin >> arr[i];
12
13     for(int i = 1; i < n; i++) {
14         int key = arr[i];
15         int j = i - 1;
16
17         while(j >= 0 && arr[j] > key) {
18             arr[j + 1] = arr[j];
19             j--;
20         }
21
22         arr[j + 1] = key;
23     }
24
25     for(int i = 0; i < n; i++)
26         cout << arr[i] << " ";
27
28     return 0;
29 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\gungu> g++ insertion_sort.cpp -o insertion_sort

C:\Users\gungu> .\insertion_sort.exe

120 320 180 100

120 180 250 320

C:\Users\gungu> |

5. Selection Sort:-

Definition:- Selection Sort repeatedly find the smallest element and place it at the beginning.

Algorithm: (step by step)

- (i) Find minimum element
- (ii) swap it with first unsorted position
- (iii) Repeat for remaining elements.

Time complexity: $O(n^2)$

Code of Selection Sort:

Problem 5: Prioritize Bug Reports:

```

#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex])
            minIndex = j;
    }
    swap (arr[i], arr[minIndex]);
}
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
return 0;
}

```

C: > Users > gungu > selection_sort.cpp > main()

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      int arr[n];
9
10     for(int i = 0; i < n; i++)
11         cin >> arr[i];
12
13     for(int i = 0; i < n - 1; i++) {
14         int minIndex = i;
15
16         for(int j = i + 1; j < n; j++) {
17             if(arr[j] < arr[minIndex])
18                 minIndex = j;
19         }
20
21         swap(arr[i], arr[minIndex]);
22     }
23
24     for(int i = 0; i < n; i++)
25         cout << arr[i] << " ";
26
27     return 0;
28 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\gungu> g++ selection_sort.cpp -o selection_sort

PS C:\Users\gungu> .\selection_sort.exe

5

9 2 8 1 5

1 2 5 8 9

PS C:\Users\gungu> |